

SCHEMA

- A **schema** is the **logical structure (design/blueprint)** of a database. It defines **how data is organized, what tables exist, their columns, data types, and relationships** between tables.
- **Schema describes the database structure, not the actual data.**

Need of Schema

1. **Organizes data**
 - Clearly defines tables and relationships
2. **Reduces redundancy**
 - Avoids unnecessary data duplication
3. **Ensures data integrity**
 - Constraints like PRIMARY KEY, FOREIGN KEY
4. **Provides data independence**
 - Changes in storage do not affect users
5. **Helps in database design**
 - Acts as a reference for developers and DBAs
6. **Security and access control**
 - Different schemas/views for different users

Example of Schema

University Database Schema

STUDENT (

StudentID INT PRIMARY KEY,

Name VARCHAR(50),

Age INT,

DepartmentID INT

)

DEPARTMENT (

DepartmentID INT PRIMARY KEY,

DepartmentName VARCHAR(50)

)

Types of Schema in DBMS

1. Physical Schema

- Describes **how data is stored physically**
- File structure, indexes, storage blocks
- Used internally by DBMS

Example: Data stored in B-tree index format on disk

2. Logical Schema

- Describes **what data is stored**
- Tables, attributes, constraints
- Designed by database designers

Example: STUDENT, COURSE, ENROLLMENT tables

3. View (External) Schema

- Describes **user-specific views**
- Shows partial data to users
- Enhances security

Example:

```
CREATE VIEW Student_View AS
```

```
SELECT Name, DepartmentID FROM STUDENT;
```

INSTANCE

- An **instance** is the **actual data stored in the database at a particular moment in time.**
- **Instance = Current state/snapshot of the database**

Example of Instance

StudentID	Name	Age	DepartmentID
101	Amit	20	1
102	Neha	21	2

COMPARISON: Schema vs Instance

Schema	Instance
Database structure/design	Actual data
Defined once	Changes frequently
Rarely changes	Changes daily
Independent of time	Time-dependent
Like blueprint	Like a real building
Created by DBA/designer	Created by users/transactions

3-TIER SCHEMA ARCHITECTURE

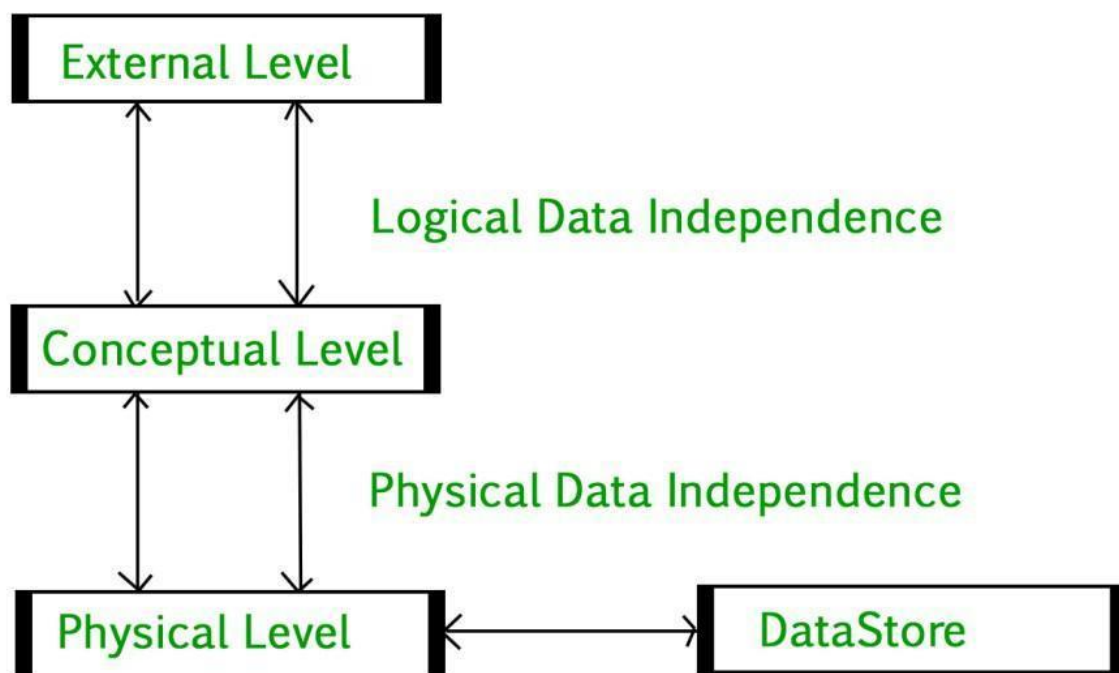
- To reduce complexity and provide **data abstraction**, DBMS follows the **ANSI/SPARC three-level architecture**.
- This architecture separates the database system into **three distinct levels (views)**:

1. External Level (View Level)
2. Conceptual Level (Logical Level)
3. Internal Level (Physical Level)

Need for Level-Wise Architecture

The level-wise architecture is required to:

- Hide complexity from users
- Support multiple user views
- Provide security and authorization
- Achieve data independence
- Improve maintainability and scalability



1. Physical Level (Internal Level)

- At the physical level, the information about the location of database objects in the data store is kept.
- The Internal Level is the **lowest level of abstraction**.
- Various users of DBMS are unaware of the locations of these objects.
- In simple terms, **the physical level of a database describes how the data is being stored in secondary storage devices** like disks and tapes and also gives insights into additional storage details.

Aspect	Details
Characteristics	• Lowest level of abstraction • Focuses on performance and storage efficiency • Hidden from end users • Uses physical storage structures
Who Uses It	• DBMS system • Database administrators (DBA)

Aspect	Details
What It Contains	• File organization methods • Indexing techniques (B-Tree, Hashing) • Record formats • Disk block allocation
Purpose	To store data efficiently and optimize access speed
Example	Data stored in disk blocks with B-Tree index on Roll No

2. Conceptual Level (LogicalLevel)

- At conceptual level, **data is represented in the form of various database tables.**
- For Example, STUDENT database may contain STUDENT and COURSE tables which will be visible to users but users are unaware of their storage.
- Also referred as **logical schema**, it describes **what kind of data is to be stored in the database.**

Aspect	Details
Characteristics	• Single conceptual schema • Independent of physical storage • Defines relationships and constraints • Central view of the database
Who Uses It	• Database designers • Database administrators (DBA)
What It Contains	• Tables (relations) • Attributes • Primary & foreign keys • Integrity constraints
Purpose	To define the complete logical structure of the database

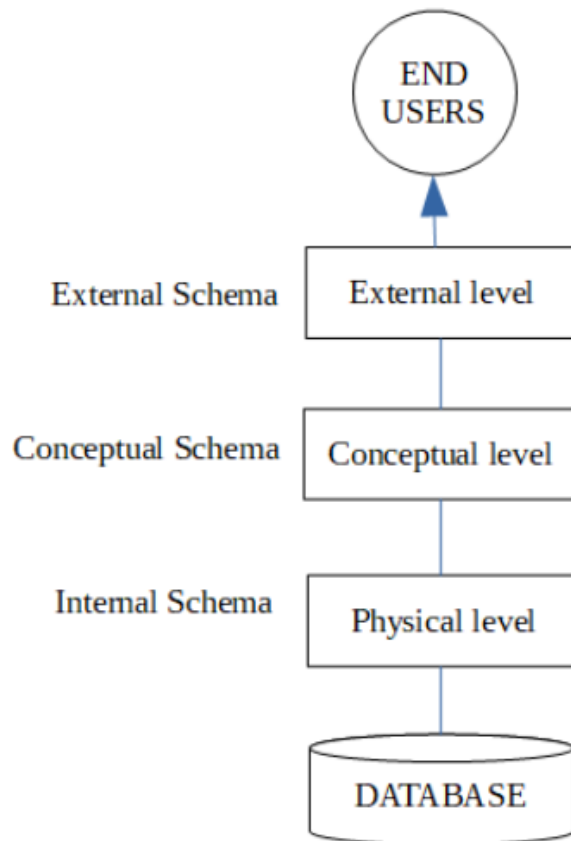
Aspect	Details
Example	STUDENT, COURSE, ENROLLMENT tables with defined relationships

3. External Level (View Level)

- An external level specifies a view of the data in terms of conceptual level tables. **Each external level view is used to cater to the needs of a particular category of users.**
- For Example, FACULTY of a university is interested in looking course details of students, STUDENTS are interested in looking at all details related to academics, accounts, courses and hostel details as well.
- So, different views can be generated for different users. The main focus of the external level is data abstraction.

Aspect	Details
Characteristics	• Highest level of abstraction • Multiple user-specific views • Hides irrelevant data • Provides security and simplicity
Who Uses It	• End users • Application programmers • Students, faculty, staff
What It Contains	• Views (virtual tables) • Selected attributes only • Derived or computed data
Purpose	To present customized data views to users and ensure data security

Aspect	Details
Example	Student view showing Roll No, Courses, Grades; Faculty view showing Attendance and Marks



DATA INDEPENDENCE

Data independence is the ability to modify the database schema at one level **without affecting the schema at the next higher level**.

DBMS provides data independence using the **three-level architecture**.

1. Logical Data Independence

Logical data independence is the ability to change the **conceptual (logical) schema without affecting external schemas (user views)**.

Aspect	Details
What Can Be Changed	• Add or remove attributes • Add new tables • Modify relationships • Change logical constraints
What Is NOT Affected	• User views • Application programs • External schemas
Level Involved	Between Conceptual Level and External Level
Example	Adding an <i>Email</i> attribute to the STUDENT table without affecting student or faculty views

Importance

- Easy database expansion
- Less impact on applications
- High flexibility

2. Physical Data Independence

Physical data independence is the ability to change the **internal (physical) schema without affecting the conceptual schema**.

Aspect	Details
What Can Be Changed	• File organization• Indexing technique• Storage location• Disk block size
What Is NOT Affected	• Table structure• Logical relationships• User views
Level Involved	Between Internal Level and Conceptual Level
Example	Changing index from Hashing to B-Tree or moving data from HDD to SSD without affecting table definitions

Importance

- Performance tuning
- Storage optimization
- Hardware upgrades

DATA MODELS

A **data model** is a set of concepts used to **describe the structure of a database**, including:

- Data types
- Data relationships
- Data constraints
- Data semantics

It defines **how data is stored, organized, and accessed** in a DBMS.

Why Data Models Are Needed?

Data models help to:

- Represent real-world entities in databases
- Define relationships among data
- Ensure data consistency

- Provide a blueprint for database design
- Support data abstraction

Types of Data Models

Data models are broadly classified into:

1. **Conceptual (High-Level) Data Models**
2. **Logical (Representational) Data Models**
3. **Physical (Low-Level) Data Models**

1. Conceptual Data Models (High-Level Models)

Conceptual data models describe **what data is required** by the organization, without considering how it will be stored.

Characteristics

- User-friendly
- Independent of DBMS
- Uses entities and relationships
- Focuses on real-world representation

Common Conceptual Models

- **Entity-Relationship (ER) Model**
- **Enhanced ER (EER) Model**

ER Model Components

- **Entities:** Real-world objects (Student, Course)
- **Attributes:** Properties of entities (Name, Roll No)
- **Relationships:** Association among entities (Enrolls)
- **Constraints:** Cardinality, participation

Example

STUDENT — enrolls — COURSE

Advantages

- Easy to understand
- Good for requirement analysis
- DBMS independent

Limitations

- Cannot be directly implemented
- No storage details

2. Logical Data Models (Representational Models)

Logical data models describe **how data is logically organized** in a database.

Example: Tables representing STUDENT, COURSE, ENROLLMENT with relationships.

Relational Data Model

- Data represented as **tables (relations)**

Key Components

- Relations (tables)
- Tuples (rows)
- Attributes (columns)
- Keys (Primary, Foreign)

Example

Roll No	Name	Dept
101	Asha	CSE

Advantages

- Simple table-based structure
- Strong data integrity using keys
- Supports powerful SQL queries
- Easy to maintain and modify

Disadvantages

- Join operations may reduce performance
- Not efficient for very complex data
- Limited support for multimedia data

3. **Physical Data Models (Low-Level Models)**

- Shows how the database is **actually implemented** in a specific DBMS.
- Contains details like **data types, indexes, storage formats, access rights**.
- Created by **DBAs and developers** for real implementation.
- Example: Tables with exact column types, indexes, and access profiles

Includes

- File structures
- Indexing
- Data compression
- Record placement

Example

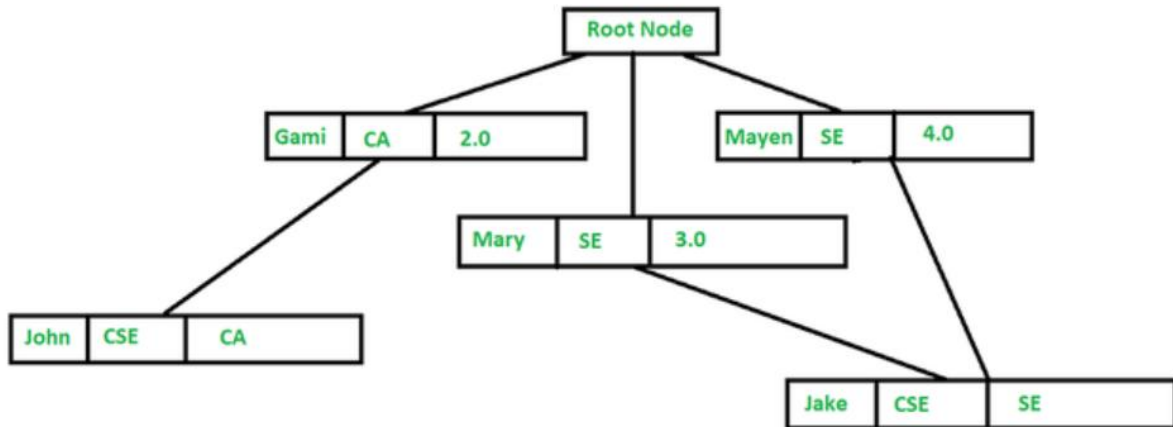
- B-Tree index, Hashing, Disk blocks

OTHER MODELS

a.) **Hierarchical Data Model**

- Data organized in **tree structure like parent-child structure**.
- One-to-many relationships

Example



Advantages

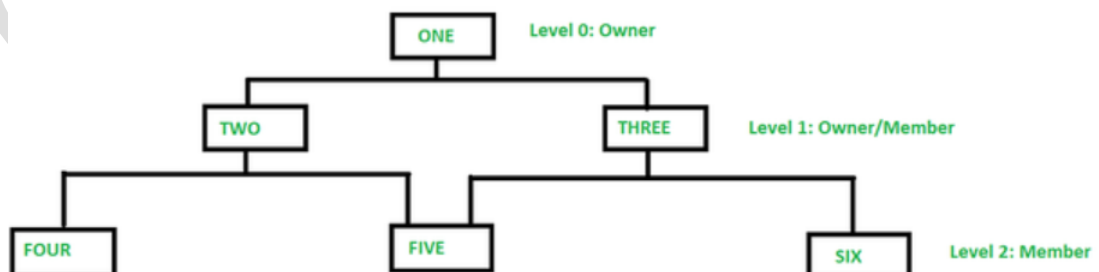
- Simple and easy to design
- Fast data access
- Good for one-to-many relationships

Disadvantages

- Does not support many-to-many relationships
- Rigid structure
- Difficult to modify

b.) Network Data Model

- Data organized as a **graph and** supports many-to-many relationships



Advantages

- Supports many-to-many relationships

- Flexible data access paths
- Faster than hierarchical model

Disadvantages

- Complex design
- Difficult to maintain
- Requires skilled programmers

c.) Object-Oriented Data Model

- **Data represented as objects with attributes and methods.**
- Supports inheritance and encapsulation

Advantages

- Supports complex data types
- Uses inheritance and encapsulation
- Good for multimedia and real-time applications

Disadvantages

- Complex implementation
- Lack of standard query language
- Less popular than relational model

Model Type	Purpose	Example
Conceptual	Requirement analysis	ER Model
Logical	Logical organization	Relational Model
Physical	Storage details	Indexing, files

Real-Life Example

- **Conceptual:** ER diagram of Student–Course–Faculty
- **Logical:** Tables created in MySQL
- **Physical:** Indexes on Roll No, storage blocks

GUNJAN CHUGH